

Puede desarrollar con su compañero de grupo. Cada ejercicio debe alzarse en la actividad VPL establecida para este laboratorio. Todos deben alzar las soluciones. Incluya el archivo de Declaración de Honor con el texto mostrado en la primera clase.

Debe tener al menos uno de los ejercicios terminados para considerar su asistencia a clase.

Lista de archivos que debe entregar en la actividad VPL

```
+-- Archivo Declaración de Honor.txt
+-- Ordenacion.java
+-- OrdenacionG.java //opcional
+-- UsoOrdenacion.java
+-- ListaEnlazadaSimple.java
+-- UsoListaEnlazadaSimple.java
```

Ambos ejercicios deben estar en la actividad.

En cada fuente (.java) debe figurar el nombre de los integrantes (y solo los que tuvieron en clase).

```
/*
    Integrantes g## : <Fulano 1>
                    <Fulano 2>
*/
```

EJERCICIO 1 – Arreglos y Generic

Implemente el algoritmo de ordenación por mezcla (MergeSort) (método llamado *ordenar*) en una clase denominada *Ordenacion*. El método *ordenar* debe recibir la referencia del arreglo a ordenar de tipo genérico. Considere cualquier tipo de elemento (Genérico). Además, la clase debe contener un método (*estaOrdenado*) que reciba un arreglo e indique si el mismo está ordenado o no (retorna un valor lógico). También debe implementar un método denominado *cntRecursos* que debe retornar en un vector la cantidad de comparaciones que hizo el método *ordenar* (recibe el arreglo a ordenar) y la cantidad de memoria utilizada en *bytes* (considere el uso de una variable estática y cuente la creación variables auxiliares utilizadas para la ordenación teniendo en cuenta la recursión - considerar el espacio simultáneo requerido).

Cree una clase *UsoOrdenacion* que utilice la clase *Ordenación* y el método *ordenar*. En esta clase debe generar aleatoriamente números entre a y b, donde a > b. Los valores para a y b deben leerse por la línea de comandos (no por la entrada estándar). Para generar números aleatorios vea *Math.random()*. En un tercer parámetro indica la cantidad de elementos a generar y un cuarto parámetro opcional que indique imprimir la cantidad de recursos utilizados (*true*: imprimir recursos).

Si la ordenación fue correcta (utilizar el método *estaOrdenado*) entonces imprimir la cantidad de segundos que tardó la ordenación. Para ello puede utilizar el método *System.currentTimeMillis()* (para más información lea el API de java). Si la ordenación no fue correcta entonces generar una excepción con el mensaje “El arreglo posee ### elementos y NO se encuentra ordenado”.

El ejemplo de funcionamiento de la clase sería (generar aleatoriamente 10000 elementos cuyos valores van de 1000 a 50000 inclusive):

```
$ java UsoOrdenacion 1000 50000 10000 true
La ordenación tardó: ### ms.
La cantidad de comparaciones fue: ###. Uso de memoria: ###b.
```

EJERCICIO 2 – Tipo Abstracto de Dato, Generic e Iteradores

Compile y pruebe la clase *ListaEnlazadaSimple* (que está disponible en el aula virtual dentro del conjunto de soluciones de la clase anterior) y complete lo que se pide. Puede probar *ListaEnlazadaSimple* en jshell (abajo un ejemplo)

Implemente:

- A. Que ahora los elementos sean comparables (debe implementar la interface *Comparable*)
- B. Que la lista se encuentre ordenada de manera ascendente (debe modificar el método **agregar**)
- C. Actualmente la operación **remover** elimina el primer elemento. Haga que reciba un parámetro y elimine todos los elementos que coincidan con el parámetro dado retornando la cantidad de eliminados.
- D. Una clase denominada **UsoListaEnlazadaSimple** que utilice *ListaEnlazadaSimple*, acepte algunos datos de la línea de comandos y los liste. Estos datos deben ser cadenas.
- E. Hacer que funcione la siguiente expresión de Java (implica uso de *java.util.iterator*) (ver la breve descripción de cómo utilizar el Iterador del API de java al final de este documento). Esta implementación realizar

```
ListaEnlazadaSimple<Integer> lista = new ListaEnlazadaSimple<Integer>();
..
for (int i : lista )
    System.out.println(i); /* Imprime cada elemento de lista */
```

Anexo - Iteración en Java - ejemplo

A continuación, tenemos la clase Cola con un iterador implementado:

```
import java.util.*;

public class Cola<Item> implements Iterable<Item> {
    private int N;
    private Nodo first;
    private Nodo last;

    private class Nodo {
        private Item dato;
        private Nodo next;
        private Nodo prev;
        ...
    }

    public Iterator<Item> iterator() {
        return new ColaIterator();
    }

    private class ColaIterator implements Iterator<Item> {
        private Node current = first;

        public boolean hasNext() { return current != null ; }
        public void remove() { throw new UnsupportedOperationException(); }

        public Item next() {
            if (!hasNext()) throw new NoSuchElementException();
            Item item = current.dato;
            current = current.next;
            return item;
        }
    }
    ...
}
```

Algunas veces el cliente necesita acceder a todos los elementos de una colección (a la cola en este ejemplo), uno a la vez, sin borrarlos. De forma a mantener el principio de encapsulación, necesitamos no revelar la representación interna de la Cola (basada en arreglos o en lista enlazada) al cliente. De esta manera necesitamos desacoplar las cosas necesarias para recorrer la lista, de los detalles de obtener cada elemento de ella. Resolvemos este desafío utilizando la interface de Java denominada: *java.util.Iterator* que mostramos a continuación:

```
public interface Iterator<Item> {
    boolean hasNext();
    Item next();
    void remove(); // opcional
}
```

Esto es, cualquier tipo de dato que implemente la interface *Iterator* “promete” implementar dos métodos: *hasNext()* y *next()*. El cliente utiliza estos métodos para acceder a la lista de elementos uno a la vez utilizando la siguiente estrategia:

```
Cola<String> queue = new Cola<String>();
queue.encolar("algo");
queue.encolar("otro");

Iterator<String> i = queue.iterator();
while (i.hasNext()) {
    String s = i.next();
    System.out.println(s);
}
```

El código de la clase Cola ilustra cómo implementar un *Iterator* cuando los elementos son almacenados en una lista doblemente enlazada. Esto descansa en una clase anidada privada denominada *Colalterator* que implementa la interface *Iterator*. El método *iterator()* crea una instancia del tipo *Colalterator* y lo retorna como un *Iterator*. Esto fuerza a la abstracción de la iteración (puesto que el cliente no conoce la implementación de *Colalterator*) y así el cliente solo accede a los elementos a través de los métodos *hasNext()* y *next()*. El cliente no tiene acceso a la parte interna de Cola y menos a *Colalterator*. La única responsabilidad del cliente es NO agregar elementos a la lista mientras el iterador se encuentre funcionando.

Mejora para la iteración. Java provee una forma más compacta para esta iteración, conocida como *enhanced for loop*. Esta forma compacta permite recorrer los elementos de una colección o de un arreglo.

Forma compacta

```
for (String s : queue)
    System.out.println(s);
```

En vez de

```
Iterator<String> i = c.iterator();
while (i.hasNext()) {
    String s = i.next();
    System.out.println(s);
}
```

Para tomar ventaja de esta forma compacta de iteración, el tipo de dato debe implementar la interface *Iterable*.

```
public interface Iterable<Item> {
    Iterator<Item> iterator();
}
```

Esto es, el tipo de dato debe implementar un método llamado *iterator()* que retorna un *Iterator* creado por la colección en cuestión. Debido a que nuestro TAD Cola ahora incluye tal método, simplemente necesitamos declararla como implementando la interface *Iterable* y estamos listos para utilizar la notación de “*foreach*”.

```
public class Cola<Item> implements Iterable<Item>
```